

CNC2024

January 23, 2025

1 Épreuve d'Informatique – Session 2024 – Filière MP

2 Correction Par Prof: EL HADIQ ZOUHAIR

2.1 Exercice: (4 points)

L'exponentiation rapide est un algorithme de complexité logarithmique $O(\log n)$, qui permet de calculer x^n :

```
Fonction puiss (x, n) :  
   $p \leftarrow 1$   
  Tant que  $n \neq 0$  faire  
    Si  $n$  est impair Alors  
       $p \leftarrow p * x$   
    Fin Si  
     $x \leftarrow x^2$   
     $n \leftarrow n/2$   
  Fin Tant que  
  Retourner  $p$ 
```

Q.1- Écrire la fonction $\text{puiss}(x, n)$ qui reçoit en paramètres un réel x , et un entier positif n . En utilisant l'algorithme de l'exponentiation rapide ci-dessus, la fonction retourne la valeur de x^n .

```
[1]: #Réponse  
def puiss(x,n):  
    p=1  
    while n!=0:  
        if n%2==1:  
            p=p*x  
        x=x**2  
        n=n//2  
    return p
```

Un polynôme à coefficients réels est représenté par une liste L : chaque élément L_i contient le coefficient du monôme de degré i .

Exemples :

- ✓ Le polynôme $7 + 4x - 3x^2 + x^5$ est représenté par la liste $[7, 4, -3, 0, 0, 1]$
- ✓ La liste $[5, -2, 0, -4, 7, 0, -2]$ représente le polynôme $-2x^6 + 7x^4 - 4x^3 - 2x + 5$

Q.2- Écrire la liste qui représente le polynôme : $-3x^3 + 7x^2 - x^6 + 10 - 2x^4$

```
[2]: #Réponse
Q2=[10,0,7,-3,-2,0,-1]
```

Q.3- Écrire la fonction valeur (x, L) qui reçoit en paramètres un réel x , et une liste L qui représente un polynôme. En utilisant la fonction puiss, la fonction retourne l'image de x par le polynôme L .

Exemple : valeur (3, [5, -2, 0, -4, 7, 0, -2]) retourne $5*3^0 - 2*3^1 + 0*3^2 - 4*3^3 + 7*3^4 + 0*3^5 - 2*3^6 = -1000$

```
[3]: #Réponse:
def valeur(x,L):
    s=0
    for i in range(len(L)):
        s+=L[i]*puiss(x,i)
    return s
```

```
[4]: #Exemples
valeur (3, [5, -2, 0, -4, 7, 0, -2])
```

```
[4]: -1000
```

Q.4- Déterminer la complexité de la fonction valeur (x, L), avec justification.

Réponse: Quasi linéaire: $O(n \log(n))$

Q.5-Écrire la fonction images (X, L) qui reçoit en paramètres une liste de réels X , et une liste L qui représente un polynôme. La fonction retourne la liste Y contenant les images des éléments de X par le polynôme L .

```
[5]: def images(X,L):
    Y=[]
    for x in X:
        Y.append(valeur(x,L))
    return Y
```

Q.6- On note le polynôme $L^2 = L \circ L$ (L rond L , 2 fois) et $L^n = L \circ L \circ L \dots \circ L$ (n fois)

Écrire la fonction compose (x, L, n) qui reçoit en paramètres un réel x , une liste L qui représente un polynôme et un entier $n > 0$. La fonction retourne l'image de x par le polynôme composé L^n .

```
[6]: #Réponse version itératif
def compose(x,L,n):
    for i in range(n):
        x=valeur(x,L)
    return x
```

```
[7]: #Réponse version récursive
def compose(x,L,n):
    if n==0:
```

```

    return x
y=compose(x,L,n-1)
return valeur(y,L)

```

```

[8]: #Réponse
def minProd(T,i,j):
    if i==j-1:
        return 0
    R=[]
    for k in range(i+1,j):
        c=minProd(T,i,k)+minProd(T,k,j)+T[i]*T[k]*T[j]
        R.append(c)
    return min(R)

```

```

[9]: T = [ 20, 10, 35, 8, 12 ]

```

```

[10]: minProd (T, 0, len(T)-1)

```

```

[10]: 6160

```

```

[11]: #Dictionnaire en parametre
def minProdTopDown(T,i,j,memo=dict()):
    if (i,j) not in memo:
        if i==j-1:
            memo[(i,j)]=0
            return 0
        R=[]
        for k in range(i+1,j):
            ↵
            ↪c=minProdTopDown(T,i,k,memo)+minProdTopDown(T,k,j,memo)+T[i]*T[k]*T[j]
            #Remarque si on oublie de faire passer memo en parametre lors de ↵
            ↪l'appel on tombe dans le cas naif
            R.append(c)
            memo[(i,j)]= min(R)
        return memo[(i,j)]

```

```

[12]: minProdTopDown (T, 0, len(T)-1)

```

```

[12]: 6160

```

```

[13]: #dictionnaire en dehors de la fonction
memo=dict()
def minProdTopDown2(T,i,j):
    if (i,j) not in memo:
        if i==j-1:
            memo[(i,j)]= 0

```

```

        return 0
        #On peut ne pas utiliser else apres return
    R=[]
    for k in range(i+1,j):
        c=minProdTopDown2(T,i,k)+minProdTopDown2(T,k,j)+T[i]*T[k]*T[j]
        #Remarque si on oublie de faire passer memo en parametre lors de
        ↳ l'appel on tombe dans le cas naif
        R.append(c)
    memo[(i,j)]= min(R)
    return memo[(i,j)]

```

```
[14]: minProdTopDown2(T, 0, len(T)-1)
```

```
[14]: 6160
```

```
[15]: #Question 9
import numpy as np

```

```
[16]: def matriceC (T):
    n=len(T)-1
    C=np.zeros((n,n),dtype=int)
    for d in range(1,n):
        #Enumerer les diagonales
        #d: Le numero du diagonale tel que j=i+d
        for i in range(n-d):
            #Enumerer les elements de chaque diagonal de numero d
            j=i+d
            C[i,j]=min([C[i,k]+C[k+1,j]+T[i]*T[k+1]*T[j+1] for k in range(i,j)])
    return C

```

```
[17]: C=matriceC(T)
C

```

```
[17]: array([[ 0, 7000, 4400, 6160],
          [ 0,   0, 2800, 3760],
          [ 0,   0,   0, 3360],
          [ 0,   0,   0,   0]])

```

```
[18]: def matriceC1(T):
    n=len(T)-1
    C=np.zeros((n,n),dtype=int)
    for d in range(1,n):
        #Enumerer les diagonales
        #d: Le numero du diagonale tel que j=i+d ou i=j-d
        for j in range(d,n):
            #Enumerer les elements de chaque diagonal de numero d
            i=j-d

```

```

        #i=j-d
        C[i,j]=min([C[i,k]+C[k+1,j]+T[i]*T[k+1]*T[j+1] for k in range(i,j)])
    return C

```

```

[19]: C1=matriceC1(T)
      C1

```

```

[19]: array([[ 0, 7000, 4400, 6160],
             [ 0,   0, 2800, 3760],
             [ 0,   0,   0, 3360],
             [ 0,   0,   0,   0]])

```

```

[20]: #Question 10

```

```

[21]: def matriceP (T, C):
      n=len(T)-1
      P=np.zeros_like(C)
      for d in range(1,n):
          for i in range(n-d):
              j=i+d
              for k in range(i,j):
                  if C[i,j]==C[i,k]+C[k+1,j]+T[i]*T[k+1]*T[j+1]:
                      P[i,j]=k
                      break
      return P

```

```

[22]: P=matriceP (T, C)
      P

```

```

[22]: array([[0, 0, 0, 0],
             [0, 0, 1, 2],
             [0, 0, 0, 2],
             [0, 0, 0, 0]])

```

```

[23]: def produit_matriciel (M,i=None,j=None,P=None):
      #deux de figure soit cette fonction doit calculer la matrice P(dans ce cas_
      →il faut passer P en argument)
      #soit utiliser comme variable global
      if i is None:
          i=0
          j=len(M)-1
          T=[M[0].shape[0]]+[m.shape[1] for m in M]
          C=matriceC (T)
          P=matriceP (T, C)
      if i==j:
          return M[i]
      if i==j-1:

```

```

        return np.dot(M[i],M[j])
    if i<j-1:
        return np.dot(produit_matriciel (M,i,P[i,j],P),produit_matriciel
↪(M,P[i,j]+1,j,P))

```

```
[24]: M=[np.random.randint(20,size=(T[i],T[i+1])) for i in range(len(T)-1)]
```

```
[25]: for m in M:
        print(m.shape)
```

```

(20, 10)
(10, 35)
(35, 8)
(8, 12)

```

```
[26]: Result=produit_matriciel (M)
```

```
[27]: import numpy as np
import matplotlib.pyplot as plt
from random import randint
```

3 Partie II : Problème

```
[28]: M=plt.imread("fleure.jpg")
```

```
[29]: def distance(X,Y):
        s=0
        for i in range(3):
            s+=abs(X[i]-Y[i])
        return s
```

```
[30]: def init_centroides (M, k):
        C=[]
        n=len(M)
        m=len(M[0])
        while len(C)< k:
            i=randint (0, n-1)
            j=randint (0, m-1)
            Pix=list(M[i,j])
            if Pix not in C:
                C.append(Pix)
        return C
```

```
[31]: C=init_centroides (M, 3)
```

```
[32]: C
```

```
[32]: [[139, 125, 238], [126, 126, 124], [14, 6, 27]]
```

```
[33]: C = [ [ 150, 160, 152 ], [ 119, 124, 118 ], [ 187, 177, 250 ] ]  
def proche_centroide (P, C):  
    return np.argmin([distance(P,c) for c in C])
```

```
[34]: proche_centroide ([137, 140, 133], C)
```

```
[34]: 1
```

```
[35]: proche_centroide ([207, 205, 255], C)
```

```
[35]: 2
```

```
[36]: proche_centroide ([154, 130, 170], C)
```

```
[36]: 0
```

```
[37]: def segmentation (M, C):  
    m,n,c=M.shape  
    R=np.zeros((m,n),dtype="int")  
    for i in range(m):  
        for j in range(n):  
            R[i,j]=proche_centroide(M[i,j],C)  
    return R
```

```
[38]: R=segmentation(M,C)
```

```
[39]: R
```

```
[39]: array([[0, 0, 0, ..., 0, 0, 0],  
        [0, 0, 0, ..., 0, 0, 0],  
        [0, 0, 0, ..., 0, 0, 0],  
        ...,  
        [1, 1, 1, ..., 1, 1, 1],  
        [1, 1, 1, ..., 1, 1, 1],  
        [1, 1, 1, ..., 1, 1, 1]])
```

```
[40]: def nv_centroide (M, R, g):  
    c=np.array([0,0,0])  
    n=0  
    for i in range(len(M)):  
        for j in range(len(M[0])):  
            if R[i,j]==g:  
                n+=1  
                c+=M[i,j]  
    return list(c//n)
```

```
[41]: nv_centroide (M, R, 0)
```

```
[41]: [148, 152, 152]
```

```
[42]: def nv_centroide2(M, R, g):  
      return list(M[R==g].mean(axis=0,dtype="int"))
```

```
[43]: nv_centroide2(M, R, 0)
```

```
[43]: [148, 152, 152]
```

```
[44]: def maj_centroides (M, R, k):  
      C=[]  
      for g in range(k):  
          C.append(nv_centroide (M, R, g))  
      return C
```

```
[45]: maj_centroides (M, R, 3)
```

```
[45]: [[148, 152, 152], [108, 114, 113], [197, 191, 237]]
```

```
[46]: #maj_centroides  
      #Pour avoir les memes valeurs que celles dans l'epreuve, il faut utiliser la  
      ↪ meme image, alors que nous avons utilisé une image capturée
```

```
[47]: def kmeans (M, k, n):  
      C0=init_centroides (M, k)  
      R=segmentation (M, C0)  
      C1=maj_centroides (M, R, k)  
      iteration=0  
      while C1!=C0 and iteration<n:  
          C0=C1  
          C1=maj_centroides (M, R, k)  
          R=segmentation (M, C1)  
          iteration+=1  
      return R,C1
```

```
[48]: R,C=kmeans (M, 3, 10)
```

```
/tmp/ipykernel_6275/1880998997.py:4: RuntimeWarning: overflow encountered in  
scalar subtract  
s+=abs(X[i]-Y[i])
```

```
[49]: C = [ [ 150, 160, 152 ], [ 119, 124, 118 ], [ 187, 177, 250 ] ]
```

```
[50]: R=segmentation(M,C)
```

```
[51]: C=maj_centroides (M, R, 3)
```



```
[52]: def calcul_inertie (M, R, C):
        inertie=0
        for i in range(len(M)):
            for j in range(len(M[0])):
                inertie+=distance(M[i,j],C[R[i,j]])
        return inertie
```

```
[53]: calcul_inertie (M, R, C)
```

```
[53]: 32065389
```

```
[54]: def inerties (M, n):
        I=dict()
        for k in range(1,10):
            R,C=kmeans (M, k, n)
            I[k]=calcul_inertie(M,R,C)
        return I
```

3.1 SQL

3.2 Q.25 La clé primaire est ligne,colonne car ces deux informations determine tous les autres champs rouge, vert blue et aussi le cluster associé

3.3 Q.26- Écrire, en algèbre relationnelle, une requête qui donne pour résultat les lignes, les colonnes, les intensités du rouge, du vert et du bleu des pixels du cluster 0 et des pixels du cluster 2.

$$\sigma_{\text{ligne,colonne,rouge,vert,bleu}}(\Pi_{\text{cluster in } (0,2)}(\text{Segmentation}))$$

3.4 Q.27- Écrire une requête SQL qui donne pour résultat les lignes, les colonnes, les intensités du rouge, du vert et du bleu des pixels du cluster 1 et des pixels du cluster 2, triés dans l'ordre croissant des lignes et colonnes.

```
SELECT ligne, colonne,rouge, vert,blue,cluster
FROM Segmentation
WHERE cluster in (1,2)
ORDER BY ligne,colonne
```

3.5 Q.28- Écrire une requête SQL qui donne pour résultat les différents clusters, le compte de pixels dans chaque cluster, l'intensité du rouge du centroïde de chaque cluster, triés dans l'ordre décroissant des comptes des pixels. L'intensité du rouge du centroïde doit être comprise entre à 100 et 200.

```
SELECT cluster,count(*) as nombre_pixels,avg(rouge) as rouge_cluster
FROM Segmentation
Group By cluster
Having rouge_cluster between 100 and 200
Order by nombre_pixels
```

```
[55]: # Remarque cette commande calcul l'intensité de rouge de la prochaine centroid_
      ↪ comme si on a ajouté une autre itération dans
      # la fonction Kmeans
```

4 Q.29- Écrire une requête SQL qui donne pour résultat la moyenne des comptes de pixels par cluster.

```
SELECT avg(number)
from (Select count(*) as number From Segmentation Group by cluster)
```